Arquitectura matemática del código

Aplicación del control óptimo de campo medio a clasificación binaria
Basado en Daudin & Delarue, *arXiv:2507.08486*

Índice

1. Datos: generación y preprocesado **2**
1.1. Generación del conjunto `make_moons` . . . 2
1.2. Normalización (`StandardScaler`) . . . 2

2. Arquitectura del modelo **2**
2.1. Campo vectorial prototípico . . . 2
2.2. Campo efectivo de campo medio . . . 3
2.3. ODE de transporte y flujo . . . 3
2.4. Integración numérica: Runge-Kutta de orden 4 . . . 4
2.5. Clasificador lineal . . . 4

3. Función de pérdida **4**
3.1. Coste total . . . 4
3.2. Entropía cruzada binaria (BCE) . . . 5
3.3. Penalización entrópica . . . 5

4. Entrenamiento **6**
4.1. Optimizador Adam . . . 6
4.2. SGD con tasa de aprendizaje constante . . . 6
4.3. SGLD precondicionado (pSGLD) . . . 6
4.4. Cosine Annealing . . . 7
4.5. Gradient Clipping . . . 8
4.6. Retropropagación . . . 8
4.7. Hiperparámetros de entrenamiento . . . 9

5. Fundamentos teóricos **9**
5.1. Condición Polyak-Łojasiewicz (PL) . . . 9
5.2. Genericidad del minimizador . . . 9

6. Descripción matemática de los experimentos **10**
6.1. Experimento A: Evolución de γ_t . . . 10
6.2. Experimento B: Efecto del parámetro ε . . . 10
6.3. Experimento C: Verificación de la condición PL . . . 10
6.4. Experimento D: Genericidad del minimizador . . . 11
6.5. Experimento E: Robustez de ν^* (make_moons) . . . 12
6.6. Experimento F: Simetría de ν^* en `make_circles` . . . 12
6.7. Experimento G: Problema de convergencia ($N \to \infty$) . . . 13

7. Resumen de la arquitectura **14**

1. Datos: generación y preprocesado

1.1. Generación del conjunto `make_moons`

Se generan N pares (x_i, y_i) con la función `make_moons` de `scikit-learn`. Cada punto pertenece a una de dos clases entrelazadas: $y_i \in \{0, 1\}$, con $N/2$ puntos por clase.

En primer lugar, se generan coordenadas limpias $x_i \in \mathbb{R}^2$: para $\theta \in [0, \pi]$, $x = (\cos\theta, \sin\theta)$ pertenece a la clase 0, y $x = (1 - \cos\theta, 1 - \sin\theta - 0{,}5)$ pertenece a la clase 1. Esto produce dos lunas entrelazadas.

A las coordenadas limpias se les añade ruido gaussiano independiente en cada dimensión:

$$x_i \leftarrow x_i + \eta_i, \qquad \eta_i \sim \mathcal{N}(0, \sigma^2 I_2), \quad \sigma = 0{,}12. \tag{1}$$

1.2. Normalización (`StandardScaler`)

Tras añadir el ruido, se aplica normalización estándar componente a componente. Sea μ_k y s_k la media y desviación típica de la componente k sobre los N puntos. La transformación es:

$$\tilde{x}_i^{(k)} = \frac{x_i^{(k)} - \mu_k}{s_k}, \qquad k = 1, 2. \tag{2}$$

Tras la normalización, cada coordenada tiene media 0 y varianza 1. Esto estabiliza el entrenamiento al colocar los datos en una escala homogénea.

En adelante se denota $x_0^i \in \mathbb{R}^2$ al punto normalizado i-ésimo y $\gamma_0 = \frac{1}{N}\sum_{i=1}^{N} \delta_{(x_0^i, y_0^i)}$ a la medida empírica de los datos de entrenamiento. La base de datos es $\{(x_0^i, y_0^i)\}_{i=1}^{N}$.

2. Arquitectura del modelo

El modelo es una *Mean-Field ResNet* compuesta de dos bloques:

1. Un **campo vectorial de campo medio** (`MeanFieldVelocity`) que define la dinámica de una ODE,
2. un **clasificador lineal** que opera sobre el estado final de la ODE.

2.1. Campo vectorial prototípico

Siguiendo el Ejemplo 1.1 del paper [1], se utiliza el campo vectorial prototípico

$$b(x, a) = \sigma(a_1 \cdot x + a_2)\, a_0, \qquad \sigma = \tanh, \tag{3}$$

donde el *parámetro de control* de cada neurona m es

$$a^m = (a_0^m,\, a_1^m,\, a_2^m,\, b_1^m) \;\in\; \mathbb{R}^2 \times \mathbb{R}^2 \times \mathbb{R} \times \mathbb{R} \;=\; \mathbb{R}^6. \tag{4}$$

Los roles de cada componente son:

- $a_1^m \in \mathbb{R}^2$: *pesos de entrada*; definen la dirección del espacio de features que la neurona m "proyecta". La pre-activación es $a_1^m \cdot x \in \mathbb{R}$.
- $a_2^m \in \mathbb{R}$ (columna temporal de W_1): coeficiente que escala la contribución explícita del tiempo $t \in [0, T]$ a la pre-activación.
- $b_1^m \in \mathbb{R}$: sesgo fijo de la neurona m en $t = 0$.
- $a_0^m \in \mathbb{R}^2$: *pesos de salida*; escalan la contribución de la neurona al campo vectorial en $\mathbb{R}^2$. Su norma $\|a_0^m\|_2$ mide la *importancia* de la neurona.

2.2. Campo efectivo de campo medio

En la aproximación de M partículas (neuronas), la medida de control $\nu_t \approx \frac{1}{M}\sum_{m=1}^{M}\delta_{a^m}$ produce el campo efectivo:

$$F(x,t) = \frac{1}{M}\sum_{m=1}^{M} b(x, a^m(t)) = \frac{1}{M}\sum_{m=1}^{M} \sigma\big(a_1^m \cdot x + a_2^m\, t + b_1^m\big)\, a_0^m. \tag{5}$$

Diferencia con el paper. El paper optimiza sobre trayectorias arbitrarias $(a^m(t))_{t\in[0,T]}$. En la implementación, los parámetros (a_1^m, a_0^m, b_1^m) son **estáticos** (independientes de t), y la dependencia temporal se introduce *explícitamente* añadiendo t como entrada adicional:

$$\text{pre-activación}^m(x,t) = a_1^m \cdot x + a_2^m \cdot t + b_1^m. \tag{6}$$

Esto restringe la familia de controles a aquellos cuya trayectoria $a_2^m \mapsto a_2^m \cdot t + b_1^m$ varía *linealmente* en t, pero preserva la dependencia temporal del campo $F(x,t)$.

Implementación matricial. Con $M = 64$ neuronas y $d_1 = 2$:

$$W_1 \in \mathbb{R}^{M\times(d_1+1)}, \quad [W_1]_{m,:} = (a_1^m,\, a_2^m), \quad b_1 \in \mathbb{R}^M, \tag{7}$$

$$W_0 \in \mathbb{R}^{d_1\times M}, \quad [W_0]_{:,m} = a_0^m \quad \text{(sin bias)}. \tag{8}$$

Para N puntos simultáneos $X \in \mathbb{R}^{N\times d_1}$:

$$h = \tanh\big(W_1\,[X,\ t\,\mathbf{1}]^\top + b_1\big) \ \in\ \mathbb{R}^{N\times M}, \qquad F(X,t) = \frac{1}{M}\, h\, W_0^\top \ \in\ \mathbb{R}^{N\times d_1}. \tag{9}$$

El factor $1/M$ se absorbe en la escala de inicialización de W_0.

Total de parámetros de la ODE.

$$\underbrace{M\cdot(d_1+1)}_{\text{pesos } W_1} + \underbrace{M}_{\text{bias } b_1} + \underbrace{d_1\cdot M}_{\text{pesos } W_0} = 64\cdot 3 + 64 + 2\cdot 64 = 384. \tag{10}$$

2.3. ODE de transporte y flujo

El estado $X_t \in \mathbb{R}^2$ de cada punto de dato evoluciona según la ODE de primer orden:

$$\frac{dX_t}{dt} = F(X_t, t), \qquad X_0 = x_0, \qquad t\in[0,T], \quad T = 1. \tag{11}$$

La solución define el *flujo* $\phi_t : \mathbb{R}^2 \to \mathbb{R}^2$ con $X_t = \phi_t(x_0)$. La distribución de features evoluciona como la *medida push-forward*:

$$\gamma_t = (\phi_t)_{\#}\gamma_0, \tag{12}$$

lo que es equivalente a la ecuación de continuidad (ec. 1.3 del paper):

$$\partial_t \gamma_t + \operatorname{div}_x\big(F(x,t)\,\gamma_t\big) = 0. \tag{13}$$

Esta PDE garantiza conservación de masa: los puntos no se crean ni se destruyen, sino que son *transportados* por el campo F.

2.4. Integración numérica: Runge-Kutta de orden 4

La ODE (11) se integra con $n_{\text{steps}} = 10$ pasos de Runge-Kutta 4 (RK4), con paso $\Delta t = T/n_{\text{steps}} = 0{,}1$. En cada paso k (con $t_k = k \cdot \Delta t$):

$$k_1 = F(X_{t_k},\ t_k), \tag{14}$$

$$k_2 = F\big(X_{t_k} + \tfrac{\Delta t}{2}\, k_1,\ t_k + \tfrac{\Delta t}{2}\big)\,, \tag{15}$$

$$k_3 = F\big(X_{t_k} + \tfrac{\Delta t}{2}\, k_2,\ t_k + \tfrac{\Delta t}{2}\big)\,, \tag{16}$$

$$k_4 = F(X_{t_k} + \Delta t\, k_3,\ t_k + \Delta t)\,, \tag{17}$$

$$X_{t_{k+1}} = X_{t_k} + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4). \tag{18}$$

El error global de RK4 es $\mathcal{O}(\Delta t^4) = \mathcal{O}(10^{-4})$, cuatro órdenes de magnitud mejor que Euler ($\mathcal{O}(\Delta t) = 0{,}1$), al coste de cuatro evaluaciones de F por paso.

2.5. Clasificador lineal

Tras integrar la ODE hasta $t = T$, se obtiene el estado final $X_T \in \mathbb{R}^{N\times 2}$. El clasificador lineal calcula el logit:

$$\ell_i = W \cdot X_T^i + b, \qquad W \in \mathbb{R}^{1\times 2},\ b \in \mathbb{R}, \tag{19}$$

y la probabilidad predicha es:

$$\hat{p}_i = \mathbb{P}(y_i = 1 \mid X_T^i) = \sigma(\ell_i) = \frac{1}{1 + e^{-\ell_i}}. \tag{20}$$

Parámetros del clasificador.

$$\underbrace{1 \times 2}_{\text{pesos } W} + \underbrace{1}_{\text{bias } b} = 3 \text{ parámetros}, \tag{21}$$

lo que representa menos del 1 % del total de parámetros del modelo (387).

Intuición. La ODE aprende a *transportar* la distribución γ_0 (lunas entrelazadas, no separables linealmente) hasta γ_T (clases separadas). El clasificador lineal simplemente lee el resultado: toda la complejidad no lineal está en el flujo.

3. Función de pérdida

3.1. Coste total

La función de pérdida que se minimiza es:

$$J(\theta) = \underbrace{\mathcal{L}_{\text{BCE}}(\theta)}_{\text{coste terminal}} + \varepsilon \cdot \underbrace{\mathcal{E}(\theta)}_{\text{penalización entrópica}}, \tag{22}$$

donde θ denota el conjunto completo de parámetros (W_1, b_1, W_0, W, b) y $\varepsilon > 0$ es el coeficiente de regularización.

3.2. Entropía cruzada binaria (BCE)

El coste terminal es la entropía cruzada binaria (Binary Cross-Entropy) con logits:

$$\mathcal{L}_{\mathrm{BCE}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \log\sigma(\ell_i) + (1-y_i)\log(1-\sigma(\ell_i))\Big], \tag{23}$$

donde $\ell_i = W \cdot X_T^i + b$ y $\sigma(\ell_i) = (1+e^{-\ell_i})^{-1}$. En términos del paper [1], este es el coste terminal $L(X_T, Y)$ del problema de control óptimo:

$$\min_{\nu_t} \mathbb{E}\big[L(X_T, Y)\big] + \varepsilon \int_0^T \cdot E(\nu_t \mid \nu^\infty)\, dt, \tag{24}$$

donde $E(\nu_t \mid \nu^\infty)$ es la divergencia KL relativa al prior ν^∞.

3.3. Penalización entrópica

Motivación teórica. El paper regula la medida de control ν_t mediante la divergencia KL respecto al prior de Gibbs:

$$\nu^\infty \propto \exp(-\ell(a)), \qquad \ell(a) = c_1\,|a|^4 + c_2\,|a|^2\,, \quad c_1 = 0{,}05,\ c_2 = 0{,}5. \tag{25}$$

El término cuártico $c_1\,|a|^4$ es esencial: satisface la condición de *supercoercividad* ($\nabla^2\ell(a) \geq c(1+|a|^2)I$) que garantiza la desigualdad log-Sobolev para ν^∞ y por tanto la condición Polyak-Łojasiewicz (véase Sección 5.1). Un potencial puramente cuadrático ($c_1 = 0$) no sería suficiente.

La KL completa es:

$$\mathrm{KL}(\nu_t \| \nu^\infty) = \underbrace{\mathbb{E}_{\nu_t}[\ell(a)]}_{\text{término de energía}} - \underbrace{H(\nu_t)}_{\text{entropía diferencial}}. \tag{26}$$

Limitación de la implementación. En la implementación, $\nu_t = \frac{1}{M}\sum_{m=1}^{M}\delta_{\theta^m}$ es una suma de deltas de Dirac sobre los parámetros puntuales. La entropía diferencial de una medida discreta respecto a un prior continuo es $-\infty$, por lo que la KL completa no es computable.

Sólo es accesible el **término de energía**:

$$\mathcal{E}(\theta) = \frac{1}{N_p}\sum_{j=1}^{N_p}\Big[c_1\,\theta_j^4 + c_2\,\theta_j^2\Big], \quad c_1 = 0{,}05,\ c_2 = 0{,}5, \tag{27}$$

donde la suma recorre los $N_p = 384$ parámetros de `MeanFieldVelocity` (se *excluyen* los 3 parámetros del clasificador, que son parte del coste terminal, no del control). Dividir por N_p hace que el valor no dependa del tamaño del modelo.

Observación 3.1. En términos prácticos, $\mathcal{E}(\theta)$ es equivalente a regularización $L^4 + L^2$ (*weight decay* polinomial). Este término proporciona el **prior energético** ν^∞, pero no el término de entropía $-H(\nu_t)$ de la KL completa. Para los experimentos B, E y F, el término de entropía se implementa mediante **SGLD** (ruido de Langevin en el gradiente, véase Sección 4.3).

Estructura de a^m para la penalización. Recordando que cada neurona m tiene parámetro $a^m = (a_0^m, a_1^m, a_2^m, b_1^m) \in \mathbb{R}^6$, la norma que aparece en $\ell(a)$ es:

$$|a^m|^2 = \|a_0^m\|^2 + \|a_1^m\|^2 + (a_2^m)^2 + (b_1^m)^2. \tag{28}$$

El término de energía es la esperanza de ℓ bajo la medida empírica $\nu_t = \frac{1}{M}\sum_{m=1}^{M}\delta_{a^m}$:

$$\mathbb{E}_{\nu_t}[\ell(a)] = \frac{1}{M}\sum_{m=1}^{M}\ell(a^m) = \frac{1}{M}\sum_{m=1}^{M}\big(c_1|a^m|^4 + c_2|a^m|^2\big).$$

Como $|a^m|^2 = \sum_j (a_j^m)^2$ descompone en escalares, esta expresión es equivalente a aplicar $c_1\theta_j^4 + c_2\theta_j^2$ a cada parámetro escalar θ_j y promediar — de ahí la forma de la ecuación (27).

Observación 3.2 (Discrepancia en el término cuártico)**.** El paper usa $|a|^4 = \left(\sum_j \theta_j^2\right)^2$; la implementación calcula $\sum_j \theta_j^4$, que omite los términos cruzados $\theta_i^2\theta_j^2$. Ambos son supercoercivos, por lo que las garantías teóricas se preservan.

4. Entrenamiento

La función `train()` implementa tres modos de optimización, cada uno adecuado a un experimento distinto:

Modo	Parámetro	Experimentos	Objetivo
Adam + cosine annealing	(defecto)	A, D	Convergencia rápida
SGD + lr constante	`use_sgd=True`	C	Verificación PL sin artefactos
pSGLD + cosine annealing	`use_sgld=True`	B, E, F, G	Muestreo de ν_t^*

4.1. Optimizador Adam

Se utiliza el optimizador Adam [4] con tasa de aprendizaje $\alpha_0 = 0{,}01$, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\varepsilon_{\text{Adam}} = 10^{-8}$. En cada paso de gradiente s, para cada parámetro escalar θ:

$$g_s = \nabla_\theta J(\theta^{s-1}), \tag{29}$$

$$m_s = \beta_1 m_{s-1} + (1-\beta_1)\, g_s \qquad \text{(primer momento, media exponencial)}, \tag{30}$$

$$v_s = \beta_2 v_{s-1} + (1-\beta_2)\, g_s^2 \qquad \text{(segundo momento, varianza exponencial)}, \tag{31}$$

$$\hat{m}_s = \frac{m_s}{1-\beta_1^s} \qquad \text{(corrección de sesgo)}, \tag{32}$$

$$\hat{v}_s = \frac{v_s}{1-\beta_2^s}, \tag{33}$$

$$\theta^s = \theta^{s-1} - \alpha_s \frac{\hat{m}_s}{\sqrt{\hat{v}_s} + \varepsilon_{\text{Adam}}}. \tag{34}$$

La corrección de sesgo es importante en los primeros pasos, cuando $m_s \approx (1-\beta_1)g_1$ y $v_s \approx (1-\beta_2)g_1^2$ son aún pequeños. Sin corrección, el paso efectivo sería demasiado grande.

4.2. SGD con tasa de aprendizaje constante

Para el Experimento C (verificación de la condición PL), se usa SGD puro con tasa de aprendizaje *constante* $\alpha = 0{,}01$ y sin ruido:

$$\theta^s = \theta^{s-1} - \alpha\, \nabla_\theta J(\theta^{s-1}). \tag{35}$$

La motivación es que la condición PL es una propiedad **geométrica** del paisaje de J, independiente del optimizador. Con cosine annealing, $\alpha_s \to 0$ al final aplana $J(\theta^s) - J^*$ artificialmente, lo que contamina la estimación de $\hat{\mu}$. Con lr constante, J^* es el mínimo geométrico real del gradient flow.

4.3. SGLD precondicionado (pSGLD)

Para los Experimentos B, E y F, se usa **pSGLD** (*preconditioned Stochastic Gradient Langevin Dynamics*, Kim et al. [3]): una extensión de SGLD que utiliza **Adam** como optimizador base y acopla el ruido de Langevin al precondicionador de Adam.

Motivación: por qué Adam como base, no SGD. En este problema los gradientes de la BCE son del orden de 10^{-5}. SGD con $\eta = 0{,}01$ produce pasos de tamaño $\sim 10^{-7}$, insuficientes para converger. Adam adapta la tasa de aprendizaje por componente mediante su precondicionador, garantizando convergencia donde SGD falla.

Fórmula de actualización pSGLD. En cada paso s, tras el paso de Adam:

$$\theta^s \leftarrow \underbrace{\mathrm{Adam}(\theta^{s-1}, \nabla J)}_{\text{paso determinístico}} + \underbrace{\sqrt{2\,\eta_s\,\varepsilon \cdot M_s}\,\xi^s}_{\text{ruido precondicionado}}, \qquad \xi^s \sim \mathcal{N}(0, I), \tag{36}$$

donde $M_s \in \mathbb{R}^{N_p}$ es el vector de precondicionadores componente a componente:

$$M_s[j] = \text{mín}\left(\frac{1}{\sqrt{\hat{v}_s[j]} + \delta},\ 1\right), \qquad \hat{v}_s[j] = \frac{v_s[j]}{1 - \beta_2^s}, \tag{37}$$

con $v_s[j]$ el segundo momento exponencial de Adam para el parámetro j, $\beta_2 = 0{,}999$ y $\delta = 10^{-8}$.

Corrección de sesgo. El término $\hat{v}_s[j] = v_s[j]/(1 - \beta_2^s)$ es el segundo momento sesgado-corregido: en los primeros pasos, $v_s[j] \approx (1 - \beta_2) g_1^2$ es pequeño aunque el gradiente no lo sea; sin corrección, $M_s[j]$ sería artificialmente grande y el ruido explotaría. La corrección garantiza que $\hat{v}_s[j] \to \mathbb{E}[g^2]$ a medida que s crece.

Cota superior en M_s y justificación. Sin la cota $M_s \leq 1$, los parámetros con dirección plana ($\hat{v}_s[j] \approx 0$) producirían $M_s[j] \approx 1/\delta = 10^8$, y el ruido $\sqrt{2\eta\varepsilon \cdot 10^8}$ explotaría incluso para ε pequeño. La cota $M_s \leq 1$ corresponde a usar un hiperparámetro de estabilización $\lambda \geq 1$ (distinto del δ de Adam) en la fórmula $1/(\sqrt{\hat{v}} + \lambda)$, como sugieren Kim et al. en su implementación original. Con esta cota, el ruido satisface $\|\text{ruido}\| \leq \sqrt{2\eta_s\varepsilon}$: nunca supera el nivel isotrópico de SGLD estándar.

Necesidad del acoplamiento ruido–precondicionador. Inyectar ruido isotrópico (σ constante para todos los j) después del paso de Adam desacopla la parte estocástica del precondicionador y destruye la distribución estacionaria: la cadena de Markov ya *no* converge a $\nu^* \propto \exp(-J(\theta)/\varepsilon)$. Con el acoplamiento de (36)–(37), la SDE efectiva es la Langevin precondicionada

$$d\theta = -\nabla J\,dt + \sqrt{2\varepsilon M}\,dW, \tag{38}$$

cuya medida estacionaria es $\nu^* \propto \exp(-J(\theta)/\varepsilon)$ bajo el precondicionador M (resultado estándar de difusiones precondicionadas).

Resultado. El ruido precondicionado implementa el **término de entropía** $-H(\nu_t)$ de la KL, haciendo que los parámetros exploren la distribución de Gibbs en lugar de colapsar al MAP. El término energético $\mathcal{E}$ en (27) sigue actuando como el prior ν^∞. Con $\varepsilon = 0$ el ruido es exactamente cero (Adam estándar).

La tasa de aprendizaje sigue cosine annealing ($\eta_s \to 0$), satisfaciendo las condiciones de Robbins-Monro: $\sum_s \eta_s = \infty$, $\sum_s \eta_s^2 < \infty$.

4.4. Cosine Annealing

La tasa de aprendizaje α_s no es constante: sigue un calendario de *cosine annealing* [5] con $\eta_{\text{mín}} = 0$ y $T_{\text{máx}} = n_{\text{epochs}}$:

$$\alpha_s = \frac{\alpha_0}{2}\left(1 + \cos\left(\frac{\pi s}{T_{\text{máx}}}\right)\right), \qquad s = 0, 1, \ldots, T_{\text{máx}}. \tag{39}$$

La tasa arranca en $\alpha_0 = 0{,}01$, decrece suavemente hasta 0 al final del entrenamiento. En Adam (experimentos A, D) y SGLD (B, E, F), el aplanamiento de las curvas de pérdida en las últimas épocas se debe al scheduler, no a una violación de la condición PL. Por eso el Experimento C usa SGD con lr constante (Sección 4.2).

4.5. Gradient Clipping

Para evitar gradientes explosivos en las primeras épocas (cuando los parámetros son aún aleatorios y el campo F puede ser grande), se aplica *gradient clipping* por norma global:

$$g \leftarrow \begin{cases} g & \text{si } \|g\| \leq G_{\text{máx}}, \\ G_{\text{máx}} \dfrac{g}{\|g\|} & \text{si } \|g\| > G_{\text{máx}}, \end{cases} \qquad G_{\text{máx}} = 5. \tag{40}$$

Aquí $g = \nabla_\theta J(\theta)$ es el vector de gradientes concatenado sobre todos los parámetros, y la norma es la euclídea global. El clipping escala el vector entero (no componente a componente), preservando la dirección del gradiente.

En Adam, el segundo momento $\hat{v}_s$ hace de divisor adaptativo, pero sin clipping la actualización puede ser muy grande en la primera época cuando $\hat{v}_s$ todavía no ha acumulado información.

4.6. Retropropagación

El gradiente $\nabla_\theta J$ se calcula automáticamente con `loss.backward()` de PyTorch. El grafo computacional incluye:

1. Diferenciación a través del clasificador lineal ($\partial J/\partial W$, $\partial J/\partial b$).
2. Diferenciación a través del RK4 (cuatro evaluaciones de F por paso, $n_{\text{steps}} = 10$ pasos): $\partial J/\partial X_T \cdot \partial X_T/\partial \theta$ mediante la regla de la cadena.
3. Diferenciación de la penalización entrópica $\partial \mathcal{E}/\partial \theta_j = 4c_1\theta_j^3 + 2c_2\theta_j$.

PyTorch construye el grafo dinámicamente en el paso *forward*, por lo que no es necesario implementar el adjunto del integrador manualmente.

4.7. Hiperparámetros de entrenamiento

Hiperparámetro	Valor	Experimentos	Descripción
N	400	todos	Puntos de entrenamiento
d_1	2	todos	Dimensión del espacio de features
M	64	todos	Neuronas / partículas
T	1.0	todos	Horizonte temporal de la ODE
n_{steps}	10	todos	Pasos RK4
ε	0.01	todos	Temperatura entrópica / regularización
α_0	0.01	todos	Tasa de aprendizaje inicial
β_1, β_2	0.9, 0.999	A, D	Momentos de Adam
n_{epochs}	800 (A), 700 (B–G)	todos	Épocas de entrenamiento
$G_{\text{máx}}$	5.0	todos	Umbral de gradient clipping
c_1	0.05	todos	Coeficiente cuártico del prior
c_2	0.5	todos	Coeficiente cuadrático del prior
Solo pSGLD (experimentos B, E, F, G):			
$\sqrt{2\eta_s \varepsilon M_s}$	variable	B, E, F, G	Escala del ruido precondicionado (por componente)
λ	1	B, E, F, G	Cota superior de M_s (evita explosión en dirs. planas)

Cuadro 1: Hiperparámetros del modelo y el entrenamiento.

5. Fundamentos teóricos

5.1. Condición Polyak-Łojasiewicz (PL)

El paper [1] demuestra (Meta-Teorema 2) que para $\varepsilon > 0$ y condiciones iniciales en un conjunto abierto y denso $\mathcal{O}$, la función de pérdida satisface la *desigualdad de Polyak-Łojasiewicz* (PL) local:

$$\|\nabla J(\theta)\|^2 \;\geq\; 2\mu \cdot (J(\theta) - J^*), \qquad \mu > 0, \tag{41}$$

donde $J^* = \inf_\theta J(\theta)$ es el valor óptimo. Esta condición implica convergencia *exponencial* del gradient descent con tasa de aprendizaje η:

$$J(\theta^s) - J^* \;\leq\; (1 - 2\eta\mu)^s \cdot (J(\theta^0) - J^*). \tag{42}$$

Observación 5.1. La condición PL es más débil que la convexidad fuerte: no implica que el espacio de parámetros sea convexo ni que el mínimo sea único en sentido geométrico, pero sí garantiza que cualquier punto estacionario es global. La clave es que $\varepsilon > 0$: con $\varepsilon = 0$ no hay garantía.

5.2. Genericidad del minimizador

El Meta-Teorema 1 del paper establece que para un conjunto abierto y denso $\mathcal{O}$ de condiciones iniciales γ_0, el problema de control tiene un único minimizador *estable*. Esto predice robustez empírica: distintas inicializaciones de parámetros y distintos datasets deberían producir el mismo J^* (a igualdad de clase de problema).

6. Descripción matemática de los experimentos

6.1. Experimento A: Evolución de γ_t

Objetivo. Visualizar que la ODE transforma γ_0 (lunas no separables) en γ_T (clases separadas).

Protocolo. Se entrena un modelo con $\varepsilon = 0{,}01$, $M = 64$, $T = 1$, $n_{\text{epochs}} = 800$. Tras el entrenamiento, se integra la ODE guardando X_t en $t \in \{0, T/4, T/2, 3T/4, T\}$. Se visualiza la distribución γ_t en cada instante.

Magnitudes visualizadas.

- Snapshots γ_t para $t \in \{0, 0{,}25, 0{,}5, 0{,}75, 1{,}0\}$.
- Trayectorias $t \mapsto X_t^{(i)}$ para 40 partículas seleccionadas.
- Curvas de convergencia: J, BCE, $\mathcal{E}$ vs. época.
- Frontera de decisión final en $\mathbb{R}^2$.

Resultado esperado. γ_T debe ser linealmente separable con accuracy $\approx 1{,}0$.

6.2. Experimento B: Efecto del parámetro ε

Objetivo. Estudiar el efecto del coeficiente de regularización entrópica sobre la convergencia, las fronteras de decisión y el campo de velocidad.

Protocolo. Se entrena un modelo para cada $\varepsilon \in \{0, 10^{-3}, 10^{-2}, 0{,}1, 0{,}5\}$ con **pSGLD** (`use_sgld=True`, Kim et al. [3]), manteniendo la misma semilla de inicialización. El uso de pSGLD es coherente con el rol de ε como temperatura entrópica: el ruido precondicionado $\sqrt{2\eta_s \varepsilon M_s}\,\xi$ hace que los parámetros exploren $\nu^* \propto \exp(-J(\theta)/\varepsilon)$. Con $\varepsilon = 0$ el ruido es exactamente cero (Adam estándar).

Magnitudes analizadas.

- **B1 — Curvas de convergencia:** J total, accuracy y $\mathcal{E}$ vs. época. Los ejes están recortados ($J \leq 0{,}5$, $\mathcal{E} \leq 0{,}15$) para que $\varepsilon = 0{,}5$ no domine la escala.
- **B2 — Fronteras de decisión y campo de velocidad:** región de decisión $\{x : \hat{p}(x) \geq 0{,}5\}$ y quiver plot de $F(x, t = 0{,}5)$ para cada ε. Fronteras más suaves y campo más regular al crecer ε.

Efecto de ε grande. Con ε pequeño ($\leq 0{,}01$), el término entrópico $\varepsilon \cdot \mathcal{E}$ es un ajuste fino: el modelo minimiza principalmente la BCE. Con $\varepsilon = 0{,}5$, la regularización pesa 50 veces más que con $\varepsilon = 0{,}01$ y se convierte en un objetivo *competitivo* con la BCE: el campo F necesita parámetros grandes para separar las lunas, pero ε los penaliza fuertemente. El resultado es un compromiso — parámetros moderados que no separan perfectamente las clases — de ahí que la accuracy no llegue a 1,0. Con $\varepsilon = 0{,}1$ el efecto es intermedio: la pérdida J no converge a cero porque el término entrópico mantiene un nivel asintótico elevado. En ambos casos el ruido pSGLD, que escala como $\sqrt{\varepsilon}$, amplifica adicionalmente las oscilaciones de las curvas.

6.3. Experimento C: Verificación de la condición PL

Objetivo. Verificar empíricamente la desigualdad (41).

Protocolo. Experimento **autocontenido**: entrena sus propios modelos con **SGD + lr constante** (`use_sgd=True`, $\varepsilon \in \{0, 10^{-3}, 10^{-2}, 0{,}1, 0{,}5\}$, 700 épocas). No reutiliza modelos del Experimento B. La motivación es que la condición PL es una propiedad **geométrica** del paisaje de

J, independiente del optimizador: con cosine annealing, $\alpha_s \to 0$ aplana $\delta_s = J(\theta^s) - J^*$ artificialmente y contamina $\hat{\mu}$. Con lr constante, J^* es el mínimo geométrico genuino del gradient flow.

En cada época s, se registran:

$$\text{excess cost}: \quad \delta_s = J(\theta^s) - J^*, \qquad \text{grad norm}: \quad G_s = \|\nabla J(\theta^s)\|^2 . \tag{43}$$

El estimador conservador de μ se define como el percentil 10 del ratio

$$\hat{\mu}_{\mathrm{PL}} = \mathrm{P}_{10}\left\{ \frac{G_s}{2\,\delta_s} : s = 1, \ldots, n_{\text{epochs}} \right\} . \tag{44}$$

Se usa el percentil 10 en lugar de la media o el mínimo porque el ratio $G_s/(2\delta_s)$ puede ser ruidoso en las épocas iniciales (cuando δ_s es grande y el gradiente aún no ha encontrado la dirección de descenso) y en las épocas finales (cuando $\delta_s \approx 0$ y el cociente diverge numéricamente). El percentil 10 descarta esos valores extremos y proporciona una cota inferior robusta: garantiza que la condición PL se cumple con constante $\hat{\mu}$ en al menos el 90 % de las épocas.

Paneles generados.

- **C1 — Log-log:** G_s vs. δ_s. Los puntos deben quedar por encima de la recta $G = 2\hat{\mu}\,\delta$. La pendiente ≈ 1 en escala log-log es la firma visual de la condición PL: $\|\nabla J\|^2$ crece proporcionalmente a $(J - J^*)$.
- **C2 — Semilog:** δ_s vs. época. Caída exponencial confirma la convergencia predicha por (41). Con SGD + lr constante el aplanamiento final refleja convergencia genuina al mínimo, sin el artefacto del cosine annealing que reduciría δ_s artificialmente al llevar $\eta_s \to 0$.
- **C3 — Barras:** $\hat{\mu}_{\mathrm{PL}}$ para cada ε. Confirma $\hat{\mu} > 0$ para todo $\varepsilon > 0$. No se garantiza que $\hat{\mu}$ crezca con ε: valores grandes de ε elevan J^*, lo que puede reducir el ratio $G_s/(2\delta_s)$.
- **C4 — Ratio PL:** $G_s/(2\delta_s)$ vs. época (escala log), que debe mantenerse $\geq \hat{\mu}$ en todo momento. Que se cumpla también en las épocas tardías confirma que la PL es una propiedad global del paisaje de J, no solo válida al inicio del entrenamiento.

6.4. Experimento D: Genericidad del minimizador

Objetivo. Verificar empíricamente el Meta-Teorema 1: que el minimizador es robusto a cambios en la inicialización y en el dataset.

Protocolo. Se ejecutan tres sub-experimentos con $n_{\text{seeds}} = 10$, 500 épocas, Adam y $\varepsilon \in \{0, 0{,}01\}$:

- **D1 — γ_0 fija, inicialización variable:** `data_seed`$= 42$, `init_seed`$\in \{0, \ldots, 9\}$.
- **D2 — Inicialización fija, γ_0 variable:** `init_seed`$= 2$, `data_seed`$\in \{0, \ldots, 9\}$.
- **D3 — Ambas aleatorias:** `data_seed`=`init_seed`$= s$.

Resultado esperado. D3 tiene la banda de pérdida más ancha (combina ambas fuentes de variabilidad). Con $\varepsilon = 0{,}01$, la condición PL garantiza $\hat{\mu} > 0$ para todo run, pero la regularización no reduce necesariamente la varianza entre seeds (el paper garantiza convergencia desde cualquier punto, no menor dispersión). Las fronteras de decisión de D2 y D3 son topológicamente similares entre sí, verificando el Meta-Teorema 1. La columna derecha de D2/D3 superpone los contornos $P(y = 1|x) = 0{,}5$ solo para runs con acc $\geq 0{,}95$, sobre una nube de puntos de todos los datasets.

Observación sobre D2. Empíricamente, la banda de D2 es casi invisible: con inicialización fija y datasets distintos, las curvas de pérdida son prácticamente idénticas entre sí. Esto se debe

a que todos los datasets generados por `make_moons` con $N = 400$ y `noise`$= 0{,}12$ son estadísticamente muy similares — distintas realizaciones de la misma distribución con suficientes puntos. La trayectoria de optimización queda determinada principalmente por θ_0, y la variabilidad específica de cada muestra es despreciable. Es la evidencia más directa del Meta-Teorema 1: lo que importa es la distribución γ_0, no la realización concreta.

6.5. Experimento E: Robustez de ν^* (make_moons)

Objetivo. Verificar que la distribución óptima de parámetros ν^* es robusta a distintas condiciones de entrenamiento: distintas inicializaciones y distintos datasets deben producir distribuciones ν^* similares.

Protocolo. Dos sub-experimentos con $n_{\text{seeds}} = 10$, $\varepsilon = 0{,}01$, 500 épocas y **pSGLD** (`use_sgld=True`, para explorar la distribución de Gibbs ν^* en lugar del MAP):

- **E-1:** `data_seed`$= 42$ fijo, `init_seed` $\in \{0, \ldots, 9\}$ variable.
- **E-2:** `init_seed`$= 2$ fijo, `data_seed` $\in \{0, \ldots, 9\}$ variable.

Magnitud visualizada. Para cada run se extraen los pesos de salida $a_0^m \in \mathbb{R}^2$ de las $M = 64$ neuronas y se calcula la *importancia*:

$$I^m = \|a_0^m\|_2 \,. \tag{45}$$

Las importancias se ordenan de mayor a menor y se representan para cada seed, junto con la media $\pm 1\sigma$ sobre las 10 seeds (figura 1×2):

- **Panel izquierdo (E-1):** importancias con init variable, datos fijos. Banda estrecha $\Rightarrow \nu^*$ es robusta a la inicialización.
- **Panel derecho (E-2):** importancias con datos variables, init fija. Banda estrecha $\Rightarrow \nu^*$ es una propiedad de la geometría del problema, no del dataset concreto.

Relación con el Experimento D y finalidad. El Experimento D mostró que J^* es robusto entre seeds: distintas condiciones de entrenamiento convergen al mismo valor de pérdida. Pero esto no implica que la *distribución* de parámetros ν^* sea la misma — dos modelos con igual J^* podrían haber llegado ahí con estructuras internas completamente distintas.

El Experimento E va un paso más allá: verifica que la estructura interna de ν^* es también robusta. Si el Meta-Teorema 1 se cumple (minimizador único estable), las curvas de importancia deben ser cualitativamente similares entre seeds: mismo "codo" pronunciado en los primeros ranks (pocas neuronas concentran la mayor parte del peso efectivo) y misma caída. Una banda estrecha en ambos paneles confirma que ν^* no es solo un mínimo con el mismo coste, sino genuinamente la *misma* distribución de parámetros.

6.6. Experimento F: Simetría de ν^* en `make_circles`

Objetivo. Verificar que en `make_circles` (dataset con simetría rotacional SO(2)) la convergencia del modelo es robusta tanto a variaciones del dataset como de la inicialización.

Protocolo. Dos sub-experimentos con $n_{\text{seeds}} = 10$, $n_{\text{epochs}} = 700$, $\varepsilon = 0{,}01$ y **pSGLD** (`use_sgld=True`):

- **F1:** `init_seed`$= 4$ fija, `data_seed` variable.
- **F2:** `data_seed`$= 42$ fijo, `init_seed` variable.

Métrica de convergencia. Con Adam o SGD el optimizador converge a un punto y J^* es el valor mínimo alcanzado — un número bien definido. Con pSGLD, en cambio, los parámetros *no* convergen a un punto: el ruido de Langevin los mantiene explorando permanentemente alrededor del mínimo. Reportar el mínimo J^* sería engañoso, porque en cualquier época el valor de J puede estar por encima por culpa del ruido, no por falta de convergencia. Por eso se usa $\bar{J}_{\text{final}}$: la media de las últimas 50 épocas, que estima el nivel estacionario de la cadena de Markov una vez que ha "llegado" a la región del mínimo y oscila alrededor de él.

Figura y conclusión. Se muestra un único panel con las curvas de convergencia $J \pm 1\sigma$ para F1 (rojo) y F2 (azul) superpuestas. Empíricamente, la curva de F1 queda casi siempre por debajo de la de F2: fijar la inicialización y variar los datos introduce menos variabilidad que fijar los datos y variar la inicialización. Esto indica que, en `make_circles`, la fuente de variabilidad dominante es la inicialización de parámetros θ_0, no el dataset concreto — resultado coherente con la observación análoga en D2 para `make_moons`.

6.7. Experimento G: Problema de convergencia ($N \to \infty$)

Objetivo. Verificar empíricamente que el valor óptimo del problema de campo medio finito J_N^* converge al valor óptimo del problema de campo medio con infinitas partículas J_∞^* cuando $N \to \infty$, y estimar la tasa de convergencia.

Marco teórico. El paper [1] trabaja en el régimen de campo medio ($M \to \infty$ neuronas) con distribución inicial *continua* γ_0. El caso discreto con N datos finitos queda fuera del alcance del paper: en la Sección 1.6 (punto 3) los autores lo señalan explícitamente como extensión futura. La convergencia $J_N^* \to J_\infty^*$ cuando $N \to \infty$ es por tanto una **conjetura** motivada por la intuición estadística, no un resultado demostrado. Una tasa natural esperada por analogía con la ley de los grandes números sería $\text{gap}(N) := J_\infty^* - J_N^* \sim C \cdot N^{-\alpha}$ con $\alpha \approx 0{,}5$. El experimento busca evidencia empírica de esta convergencia y estima la tasa α.

Protocolo. Se consideran $N \in \{25, 50, 100, 200, 400, 800\}$ partículas: una progresión geométrica de razón 2 que cubre dos órdenes de magnitud, desde un régimen de datos escasos ($N = 25$, apenas suficientes para definir las dos lunas) hasta un régimen rico ($N = 800$, donde la distribución empírica aproxima bien a γ_0). El valor $N = 400$ es el estándar del resto de experimentos, lo que permite comparar directamente. Se eligen $\varepsilon \in \{0, 0{,}01\}$ para contrastar el caso sin regularización entrópica con el caso de referencia del paper. Para cada combinación (N, ε) se realizan 20 entrenamientos independientes (`n_seeds`= 20) con 700 épocas y **pSGLD** (`use_sgld=True`). Se usa pSGLD (no Adam) para que el nivel de ruido sea coherente con ε: con $\varepsilon = 0$ el ruido es exactamente cero. Las 20 semillas son necesarias para estimar la media y la varianza del gap con fiabilidad estadística.

Estimador de J_∞^*. El valor J_∞^* es inaccesible directamente; se aproxima mediante un *test set oráculo*: se evalúa la pérdida BCE (sin término de regularización) del modelo entrenado sobre un conjunto de test de $N_{\text{test}} = 10\,000$ puntos i.i.d. de make_moons (semilla fija = 999). La hipótesis es que, para N grande, el BCE en test tiende a J_∞^* (la pérdida de Bayes en la distribución de datos). El gap se define como:

$$\text{gap}(N) := J_\infty^* - J_N^* \;\approx\; \text{BCE}_{\text{test}}(N_{\text{large}}) - \text{BCE}_{\text{test}}(N). \tag{46}$$

En la práctica, J_∞^* se aproxima con la media de BCE_test sobre las 20 semillas para $N = 800$.

Figura (G_convergence_problem.png). Layout 1×3:

- **G1 — Convergencia de la pérdida con N:** BCE_test (línea continua) y BCE_train (línea discontinua), media $\pm 1\sigma$ sobre 20 seeds, vs. N en escala log en x, para $\varepsilon = 0$ y $\varepsilon = 0{,}01$. Evidencia visual de que BCE_test$(N) \to J_\infty^*$ al crecer N. La regularización entrópica ($\varepsilon = 0{,}01$) queda por debajo de $\varepsilon = 0$ para N pequeño: actúa como prior que compensa la falta de datos.

- **G2 — Gap de generalización:** $\mathrm{gap}(N) = \mathrm{BCE_test} - \mathrm{BCE_train}$ vs. N. Para N pequeño el modelo sobreajusta (BCE_train baja pero BCE_test alta), de ahí el gap grande. Al crecer N el gap se reduce porque la distribución empírica aproxima mejor a γ_0 y hay menos sobreajuste.

- **G3 — Tasa de convergencia del gap (log-log):** $\log \mathrm{gap}(N)$ vs. $\log N$ con ajuste lineal (mínimos cuadrados sobre $N \geq 100$), cuya pendiente es $-\alpha$. Se excluyen $N < 100$ porque en el régimen escaso el gap no sigue aún la ley de potencias asintótica. Los resultados empíricos son: $\varepsilon = 0$: $\alpha \approx 0{,}78$; $\varepsilon = 0{,}01$: $\alpha \approx 0{,}85$. Ambas tasas son más rápidas que la referencia $N^{-0{,}5}$.

  Cálculo de α. Se asume que el gap sigue una ley de potencias $\mathrm{gap}(N) \propto N^{-\alpha}$. Tomando logaritmos, $\log \mathrm{gap} = -\alpha \log N + c$, que es una recta en escala log-log. El exponente α se estima como la pendiente (con signo cambiado) de la regresión lineal por mínimos cuadrados ordinarios sobre los pares ($\log N$, $\log \mathrm{gap}$) con $N \geq 100$.

  Referencia clásica $N^{-1/2}$. La tasa $\alpha = 0{,}5$ corresponde a la velocidad de convergencia de la medida empírica $\hat{\gamma}_N$ hacia la medida continua γ_0 garantizada por el **teorema de Glivenko-Cantelli**: bajo hipótesis estándar de momentos, la distancia de Wasserstein (o la norma supremo del proceso empírico) entre $\hat{\gamma}_N$ y γ_0 es $O(N^{-1/2})$. Esta cota se usa aquí como referencia natural para juzgar si la regularización entrópica acelera o no la convergencia.

7. Resumen de la arquitectura

Bloque	Descripción
Datasets	make_moons (A–E, G) y make_circles (F), $N = 400$, normalizados
ODE	$dX_t/dt = F(X_t, t)$, $t \in [0, 1]$, 10 pasos RK4
Campo F	$(1/M) \sum_m \tanh(a_1^m \cdot x + a_2^m t + b_1^m)\, a_0^m,\ \ M = 64$
Parámetros ODE	$W_1 \in \mathbb{R}^{64\times 3}$, $b_1 \in \mathbb{R}^{64}$, $W_0 \in \mathbb{R}^{2\times 64}$ (384 total)
Clasificador	$\ell = W \cdot X_T + b,\ \ W \in \mathbb{R}^{1\times 2},\ \ b \in \mathbb{R}$ (3 total)
Total parámetros	387
Pérdida	$J = \mathrm{BCE} + \varepsilon \cdot \mathcal{E},\ \ \varepsilon = 0{,}01$
Optimizador (A, D)	Adam [4], $\alpha_0 = 0{,}01$, cosine annealing
Optimizador (C)	SGD, $\alpha = 0{,}01$ constante, sin ruido
Optimizador (B, E, F, G)	pSGLD [3], $\theta \leftarrow \mathrm{Adam}(\theta, \nabla J) + \sqrt{2\eta\varepsilon M}\,\xi,\ \ M = \mathrm{mín}(1/\sqrt{\hat{v}} + \delta, 1)$
Gradient clip	$G_{\mathrm{máx}} = 5$ (todos los modos)

Referencias

[1] S. Daudin and F. Delarue, *Genericity of the Polyak-Łojasiewicz inequality for mean-field Neural ODEs with entropic regularization*, arXiv:2507.08486, 2025.

[2] M. Welling and Y. W. Teh, *Bayesian Learning via Stochastic Gradient Langevin Dynamics*, in *International Conference on Machine Learning (ICML)*, 2011.

[3] S. Kim, Q. Song, and F. Liang, *Stochastic Gradient Langevin Dynamics Algorithms with Adaptive Drifts*, arXiv:2009.09535, 2020.

[4] D. P. Kingma and J. Ba, *Adam: A Method for Stochastic Optimization*, in *International Conference on Learning Representations (ICLR)*, 2015. https://arxiv.org/abs/1412.6980

[5] I. Loshchilov and F. Hutter, *SGDR: Stochastic Gradient Descent with Warm Restarts*, in *International Conference on Learning Representations (ICLR)*, 2017. https://arxiv.org/abs/1608.03983

[6] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, *Neural Ordinary Differential Equations*, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. https://arxiv.org/abs/1806.07366

[7] B. T. Polyak, *Gradient methods for the minimisation of functionals*, *USSR Computational Mathematics and Mathematical Physics*, 3(4):864–878, 1963.

[8] S. Łojasiewicz, *Une propriété topologique des sous-ensembles analytiques réels*, in *Colloques Internationaux du CNRS*, 117:87–89, 1963.